

13장 포인터와 배열

1. 포인터와 배열의 관계
2. 포인터 연산
3. 상수형태의 문자열을 가리키는 포인터
4. 포인터 배열



문자열과 포인터

3

- 문자열을 저장하는 방법
 - ▣ 문자형 배열을 이용하는 방법
 - ▣ 문자형 포인터에 문자열의 주소를 저장하는 방법
 - ▣ 2가지 방법 모두 마지막에 모두 널문자를 자동으로 저장해줌

```
char s[ ] = "HelloWorld";           // ①  
char* p = "HelloWorld";             // ②
```

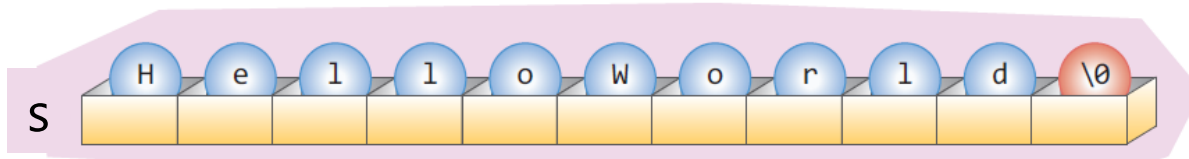


문자형 배열을 이용하는 방법

4

- 문자형 배열을 위한 메모리 공간이 할당
- 배열은 변수이므로 값의 변경이 가능함
- 컴파일러는 초기값으로 주어진 문자열상수를 문자상수 표현으로 변환하여 배열을 초기화함

```
char s[] = "HelloWorld";  
=> char s[] = {'H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', '\0'};
```



데이터 세그먼트
(값을 변경할 수 있는 메모리 영역)

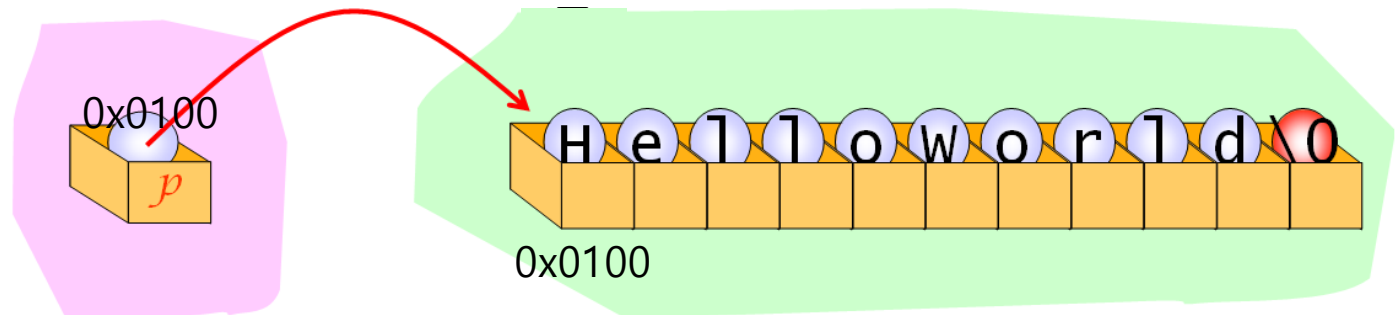
11바이트 변수공간(변경 가능)



문자열 포인터를 이용하는 방법

- char*형의 포인터 변수를 선언 -> 4 or 8바이트 공간할당
- 문자열 상수는 상수를 저장하는 메모리 공간에 자동 저장됨
- 문자열 상수는 저장된 메모리의 시작 주소로 치환됨
- 포인터 변수에는 문자열 상수가 저장된 주소값이 저장됨

```
char* p = "HelloWorld";  
=> char* p = (char*)0x0100;
```



8바이트 변수공간(변경 가능)

11바이트 상수공간(변경 불가능)



문자열과 포인터

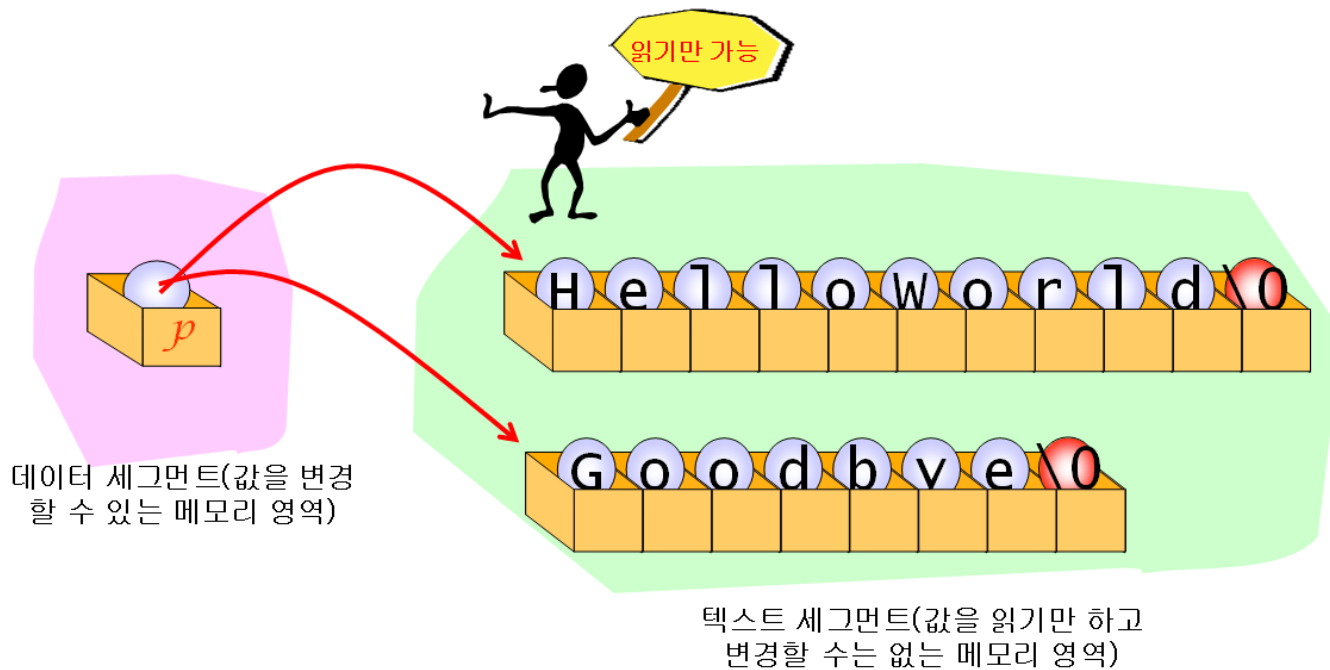
□ 문자열의 값을 변경하는 방법

```
char s[] = "HelloWorld";  
s = "Goodbye";           // 오류  
strcpy(s, "Goodbye");     // 정상, 21장에서 배움  
  
char *p = "HelloWorld";  
strcpy(p, "Goodbye");     // 오류  
p = "Goodbye";           // 정상
```



문자열과 포인터

- 포인터 p 는 변수이므로 다른 문자열의 주소를 저장 가능함





복습: 정수 입출력

8

- 정수 입력 : 서식문자 %d 이용, 정수형 변수의 **주소를 전달**해야 함
- 정수 출력 : 서식문자 %d 이용, 정수형 변수의 **값을 전달**해야 함

```
int x;  
printf("문자를 입력하시오:");  
scanf("%d", &x);  
printf("%d\n", x);
```

정수를 입력하시오: 10 <엔터>
10



복습: 문자 입출력

9

- 문자 입력 : 서식문자 %c 이용, 문자형 변수의 **주소를 전달**해야 함
- 문자 출력 : 서식문자 %c 이용, 문자형 변수의 **값을 전달**해야 함

```
char ch;  
printf("문자를 입력하시오:");  
scanf("%c", &ch);  
printf("%c\n", ch);
```

문자를 입력하시오: H <엔터>
H



복습: 문자열 입출력(11장)

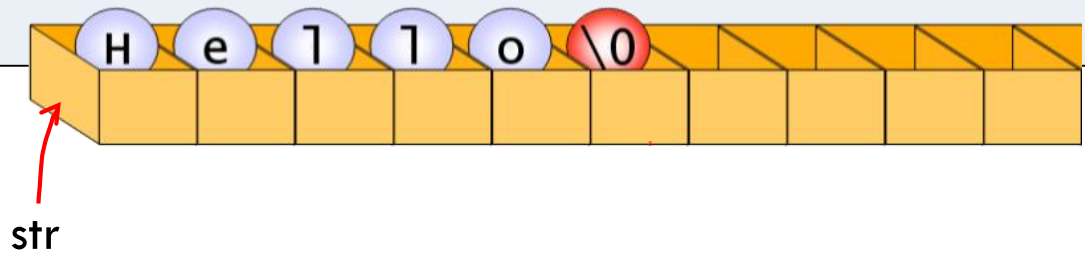
10

- 문자열 입력 : 서식문자 %s 이용, 문자 배열의 **시작주소를 전달**해야 함 -> 배열명 앞에 &를 붙이지 않음, scanf함수가 널문자를 자동으로 저장해줌
- 문자열 출력 : 서식문자 %s 이용, 문자 배열의 **시작주소를 전달**해야 함-> 배열이름 str의 앞에는 & 연산자를 붙이지 않음, 널문자가 나올 때까지 출력,

```
char str[10];  
  
printf("문자열을 입력하시오:");  
scanf("%s", str);  
printf("%s\n", str);
```

문자열을 입력하시오: Hello <엔터/>
Hello

// &str -> 오류
// Hello 출력





예제 1 : 문자열과 포인터

11

```
#include <stdio.h>
int main(void)
{
    char s[] = "HelloWorld";
    char* p = "HelloWorld";

    printf("문자형 배열에 저장된 문자열 = %s\n", s);
    printf("포인터가 가리키는 문자열 = %s\n", p);
    p = "Goodbye";        // 이것은 가능하다.
    // s = "Goodbye";     // 컴파일 에러임
    printf("포인터가 가리키는 문자열 = %s\n", p);
    return 0;
}
```

문자형 배열에 저장된 문자열 = HelloWorld
포인터가 가리키는 문자열 = HelloWorld
포인터가 가리키는 문자열 = Goodbye



예제 2: 문자열과 포인터

12

```
#include <stdio.h>
int main(void)
{
    char* p = "HelloWorld";
    printf("%s\n", p);
```

```
    p = "Welcome to C World!"; // 가능
    printf("%s\n", p);
```

```
    p = "Goodbye"; // 가능
    printf("%s\n", p);
```

```
    // p[0] = 'a';
```

```
    return 0;
```

```
}
```

```
HelloWorld
Welcome to C World!
Goodbye
```

// 오류가 발생한다.

*포인터 p의 값을 변경하는 것은 가능하나
p[0]는 'G'가 저장된 상수공간을 의미함*



예제 3: 문자열과 포인터

13

```
#include <stdio.h>
int main(void)
{
    char str1[] = "My String";
    char* str2 = "Your String";
    printf("%s %s \n", str1, str2);
```

```
    str2 = "Our String";    // 가리키는 대상 변경
    printf("%s %s \n", str1, str2);
```

```
    str1[0] = 'X';    //가능
    //str2[0] = 'X';    //불가능
    printf("%s %s \n", str1, str2);
    return 0;
```

```
}
```

My String Your String
My String Our String
Xy String Our String

*포인터 str2의 값을 변경하는 것은 가능하나
str2[0]는 'O'가 저장된 상수공간을 의미함*



문자열과 포인터

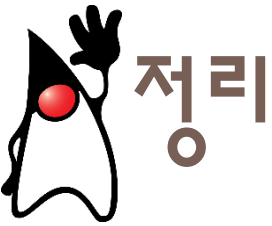
14

- 문자배열의 초기값을 제외하고 모든 문자열 상수는 주소값으로 치환됨

```
char* str = "Your String";  
=> char* str = (char*)0x1234;
```

```
printf("Show your string");  
=> printf((char*)0x1234);
```

```
WhoAreYou("Hong");  
=> WhoAreYou((char*)0x1234);  
=> void WhoAreYou(char* str){...};
```



- 일반적으로 문자열을 저장할 때는 문자형 배열(char str[])을 사용
- 문자형 포인터(char* ptr)는 문자열 상수를 저장할 때만 사용가능 -> 함수의 매개변수에 주로 사용됨, 21장에서 다룸



포인터 배열

16

- 포인터 배열(array of pointers): 포인터(주소)를 모아서 배열로 만든 것
- 배열의 요소가 포인터(주소)임을 의미
- 일반 배열의 선언과 차이점은 변수의 자료형을 표시하는 위치에 `int` 나 `double`을 대신해서 `int*` 나 `double*` 가 올 뿐임

```
자료형* 배열명[배열크기];  
-> int* arr[10];
```



포인터 배열

17

```
char* arr1[5];
```

//char*형 포인터를 모아 놓은 것

```
int* arr2[5];
```

//int*형 포인터를 모아 놓은 것

```
double* arr2[5];
```

//double*형 포인터의 배열

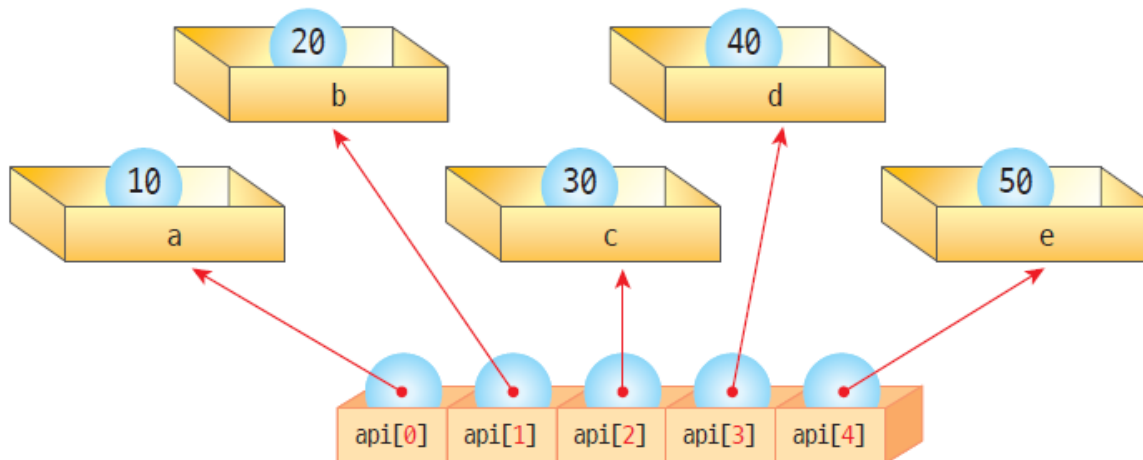


정수형 포인터 배열

18

- 정수를 가리키는 주소를 모아 놓은 배열

```
int a = 10, b = 20, c = 30, d = 40, e = 50;  
int* api[5] = { &a, &b, &c, &d, &e };
```





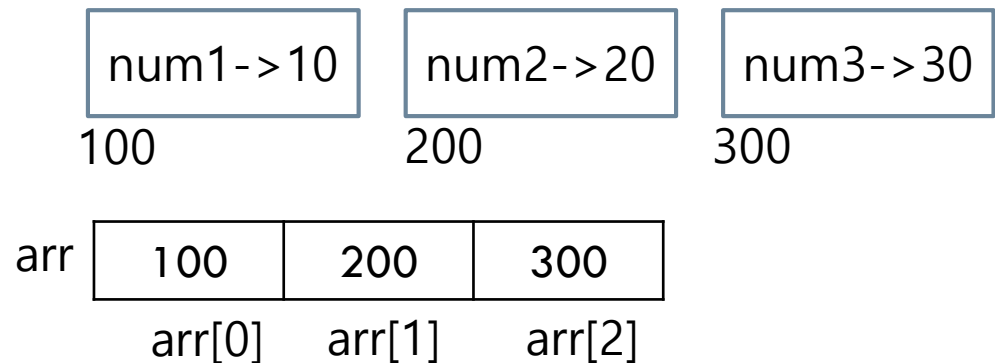
예제 4: 정수형 포인터 배열

19

```
#include <stdio.h>
int main(void)
{
    int num1 = 10, num2 = 20, num3 = 30;
    int* arr[3] = { &num1, &num2, &num3 };

    printf("%d\n", *arr[0]); //arr[0]는 주소, *arr[0]->*주소->값
    printf("%d\n", *arr[1]);
    printf("%d\n", *arr[2]);
    return 0;
}
```

10
20
30





예제 5: 정수형 포인터 배열

20

```
#include <stdio.h>
int main(void)
{
    int i, n;
    int a = 10, b = 20, c = 30, d = 40, e = 50;
    int* pa[5] = { &a, &b, &c, &d, &e };
    n = sizeof(pa)/sizeof(pa[0]);    // 배열 원소 개수 계산
    for(i = 0; i < n; i++)
        printf("%d\\n", *pa[i]);    // pa[i]는 주소, *pa[i]는 값

    return 0;
}
```

10
20
30
40
50



문자열 포인터 배열

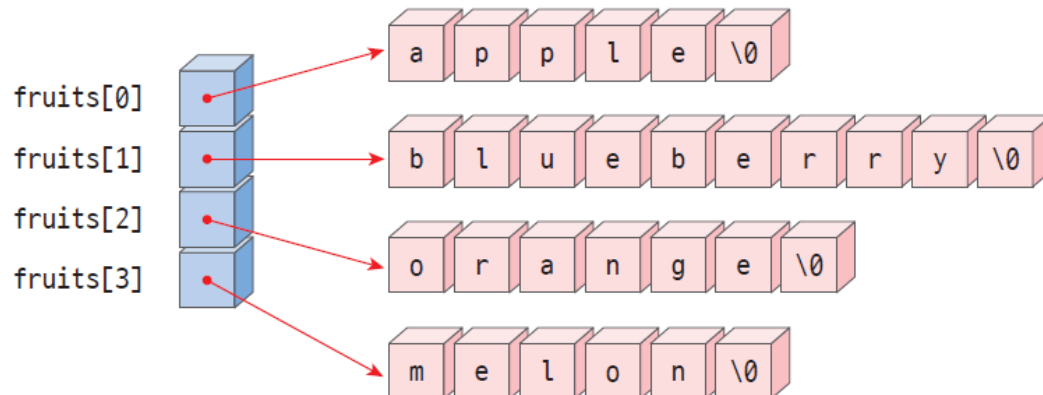
21

- 문자열의 주소를 모아 놓은 배열
- 문자열상수는 메모리 공간에 저장된 후 그것의 주소값으로 대체됨

```
char* fruits[ ] = {  
    "apple",  
    "blueberry",  
    "orange",  
    "melon"  
};
```



```
char* fruits[ ] = {  
    0x1000,  
    0x2000,  
    0x3000,  
    0x4000  
};
```





예제 6: 문자형 포인터 배열

22

```
#include <stdio.h>
int main(void)
{
    char* strArr[3] = { "Simple", "String", "Array" };

    printf("%s\n", strArr[0]);
    printf("%s\n", strArr[1]);
    printf("%s\n", strArr[2]);
    return 0;
}
```

Simple
String
Array



예제7: 문자형 포인터 배열

23

```
#include <stdio.h>
int main(void)
{
    int i, n;
    char* fruits[ ] = {
        "apple",
        "blueberry",
        "orange",
        "melon"
    };

    n = sizeof(fruits)/sizeof(fruits[0]);
    for(i = 0; i < n; i++)
        printf("%s\n", fruits[i]);

    return 0;
}
```

apple
blueberry
orange
melon

// 배열 원소 개수 계산



실습과제 1

24

- C언어에서 문자를 메모리에 저장하는 방식에 대하여 설명하라(아스키 코드 참고)
- 널문자의 용도를 설명하라.
- 메모리에 저장된 문자열의 끝을 찾는 방법을 설명하라.
- 문자열의 길이를 구하는 알고리즘을 설명하라.

- 키보드를 통하여 문자열을 입력 받고 문자열의 각 문자를 실행 결과처럼 출력하시오.
- 문자열의 끝은 널문자('\0')를 이용하여 판단할 것

문자열을 입력하시오:HelloWorld <엔터>

1번째문자 H

2번째문자 e

3번째문자 l

4번째문자 l

5번째문자 o

6번째문자 W

7번째문자 o

8번째문자 r

9번째문자 l

10번째문자 d



실습과제3

26

- 키보드를 통하여 문자열을 입력 받고 대문자는 소문자로 소문자는 대문자로 변환하는 코드를 작성하라.
- 문자열의 끝은 널문자('\0')를 이용하여 판단할 것, 라이브러리 함수를 사용하지 말고 직접 코드로 작성할 것

문자열을 입력하시오:HelloWorld <엔터>
변환결과 : hELLOwORLD

- 2개의 문자열(모두 소문자)을 입력 받고 사전에서 앞에 나오는 문자열을 출력하시오.
- 라이브러리 함수를 사용하지 말고 문자열의 첫번째 문자의 아스키코드를 비교하여 구할 것
- 사전에서 앞에 나오는 문자가 아스키코드가 작음

문자열을 입력하시오:hello<엔터>
문자열을 입력하시오:world<엔터>
사전에서 앞에 나오는 문자열: hello



실습과제 5

28

- 다음 코드에서 사전에서 제일 앞에 나오는 문자열을 찾는 코드를 작성하시오.
- 함수를 사용하지 말고 각 문자열의 첫번째 문자중에서 최소값을 찾아라.

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    char* fruits[ ] = {"apple", "blueberry", "orange", "melon"};
```

```
    // 사전에서 제일 앞에 나오는 문자열을 찾는 코드
```

```
    return 0;
```

```
}
```

사전에서 가장 앞에 나오는 문자열: apple



과제제출방법

29

- 아래 주소의 작성방법대로 github에 제출할 것
 - ▣ <https://github.com/2sungryul/cprogramming/tree/main>
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```